

LAB01b Zeri, minimi locali, vincoli box

Antonio Scala

10 Mar 2026

Materiale di partenza: i tre script

- 01_zero_newton_vs_bisection.py
- 02_minimi_two_basins_multistart.py
- 03_box_constraint_penalty_vs_projection.py

Obiettivo: (i) capire *perché* certi metodi falliscono o sono fragili, (ii) imparare a leggere la diagnostica (residuo, passo, iterazioni), (iii) confrontare con chiamate “industriali” ai solutori standard (SciPy).

Nota pratica: in Python i solutori “pronti all’uso” per zeri e ottimizzazione non stanno in NumPy ma in **SciPy**, nel modulo `scipy.optimize`. In particolare:

- zeri 1D -> `scipy.optimize.root_scalar`
- minimi 1D -> `scipy.optimize.minimize_scalar` / `fminbound`
- ottimizzazione con vincoli box -> `scipy.optimize.minimize(..., bounds=...)`.

Sistemi lineari: usare **NumPy** (`numpy.linalg`) per matrici dense; usare **SciPy** (`scipy.linalg`, `scipy.sparse.linalg`) quando servono fattorizzazioni/solutori più robusti o matrici sparse.

0) Setup

1. Eseguire gli script così come sono, senza modificarli, e annotare:
 - esito (converge/non converge);
 - iterazioni;
 - valori finali e messaggio/diagnostica stampata.
2. Se manca SciPy (per la parte di confronto), installarlo:
 - `pip install scipy` (o via conda, se usate conda).

1) Esercizio A – Zeri: Newton “alla cieca” vs bracketing

A1. Osservazione guidata sul programma `01_zero_newton_vs_bisection.py` (10 min)

- Identificare:
 - $f(x)$ e $f'(x)$;
 - il bracket usato per la bisezione;
 - l’inizializzazione di Newton.
- Domande:
 1. perché la bisezione è garantita (quale proprietà su $f(a), f(b)$)?
 2. cosa succede alla sequenza di Newton (guardare gli ultimi iterati stampati)? È un ciclo? Diverge?

A2. Piccole modifiche “controllate”

- Cambiare *solo* x_0 di Newton (es. $x_0=-1.0$, $x_0=-1.5$, $x_0=-0.5$) e vedere quando converge.
- Cambiare il bracket (es. $[-2,0]$ vs $[-1,0]$) e verificare cosa succede se *non* c’è cambio di segno.

A3. Confronto con SciPy: `root_scalar` e metodi bracketing

Aggiungere (o creare uno script minimo a parte) e provare:

```
from scipy import optimize

def f(x):
    return x**3 - 2*x + 2

# Metodo robusto di bracketing: Brent (consigliato in 1D se hai un bracket)
sol = optimize.root_scalar(f, bracket=(-2, 0), method="brentq")
print(sol.root, sol.iterations, sol.converged)

# Bisezione "pura"
sol_bi = optimize.root_scalar(f, bracket=(-2, 0), method="bisect")
print(sol_bi.root, sol_bi.iterations, sol_bi.converged)

# Newton: richiede x0 e (idealmente) fprime
def df(x):
    return 3*x**2 - 2

sol_n = optimize.root_scalar(f, x0=0.0, fprime=df, method="newton")
print(sol_n.root, sol_n.iterations, sol_n.converged)
```

Cosa notare:

- `brentq` è “safer” ma richiede bracket; è spesso la scelta standard in 1D se lo hai. (docs.scipy.org)

- `root_scalar` ritorna un oggetto con `root`, `iterations`, `converged` (diagnostica “pulita”). ([docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize.minimize_scalar.html))
- sintassi per passare funzioni:
 - definizione con `def`;
 - oppure `lambda x: ...` (ma meglio `def` per leggibilità).

Mini-task: replicare nel confronto SciPy un caso in cui Newton fallisce con `x0=0.0` ma `brentq` converge.

2) Esercizio B – Minimi locali: single-start vs multi-start

B1. Leggere `02_minimi_two_basins_multistart.py`

- Identificare:
 - obiettivo $f(x)$ e derivata $f'(x)$;
 - parametri di discesa del gradiente (step `alpha`, criterio su `gtol`).
- **Domande:**
 1. dov'è (circa) ciascun minimo locale?
 2. perché il risultato dipende da `x0_single`?

B2. Esperimenti rapidi

- Provare diversi `x0_single` e annotare a quale minimo converge.
- Cambiare `alpha` (es. 0.01, 0.05) e osservare: convergenza più lenta vs rischio di oscillazioni/divergenza.

B3. Confronto con SciPy: `fminbound` / `minimize_scalar`

Per minimi 1D, usare Brent bounded:

```
from scipy import optimize

def f(x):
    return (x + 1.0)**2 * (x - 2.0)**2 + 0.10*x

# Minimo in un intervallo (bounded), metodo tipo Brent
xopt = optimize.fminbound(f, -4.0, 4.0)
print("xopt =", xopt, "f(xopt) =", f(xopt))
```

`fminbound` usa Brent per minimi in un intervallo. ([docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize.minimize_scalar.html))

Task concettuale: confrontare `xopt` (bounded) con gli esiti del multi-start: * se il multi-start ha trovato un minimo peggiore, discutere “copertura” dei punti iniziali e step size; * se coincide, discutere perché bounded ha un vantaggio (esplora l'intervallo in modo sistematico in 1D).

3) Esercizio C – Vincolo box: penalità vs proiezione

C1. Leggere 03_box_constraint_penalty_vs_projection.py

- Identificare:
 - box $[a, b]$;
 - minimo non vincolato (qui è fuori dal box);
 - come è definita la penalità e come funziona la proiezione (`clip`).

Domande: 1. qual è l'ottimo vincolato atteso? Perché sta sul bordo? 2. perché nel metodo proiettato non ha senso aspettarsi $\nabla f(x^*) = 0$?

C2. Esperimenti (10 min)

- Cambiare `mu` nella penalità (10, 50, 200) e osservare: rispetto del vincolo vs stabilità/rigidità.
- Cambiare `alpha` nel projected GD e verificare se converge “più pulito” o oscilla.

C3. Confronto con SciPy: minimize con bounds

Esempio minimale con box:

```
from scipy import optimize
```

```
def f(x):  
    # x arriva come array in minimize  
    return (x[0] - 3.0)**2
```

```
x0 = [1.5]  
bounds = [(0.0, 2.0)]
```

```
res = optimize.minimize(f, x0, method="L-BFGS-B", bounds=bounds)  
print(res.x, res.fun, res.success, res.message)
```

- `bounds=[(a,b), ...]` è l'interfaccia standard per vincoli box in `minimize` con metodi che li supportano (es. L-BFGS-B). (docs.scipy.org)

Task: verificare che `res.x` finisca sul bordo 2.0 e confrontare con penalità/proiezione.

4) Consegna finale

Ogni gruppo consegna (anche solo oralmente + appunti):

1. Un esempio concreto di fallimento o fragilità di Newton (cosa lo causa, come lo “salvi”).
2. Un esempio concreto di dipendenza da inizializzazione nei minimi locali (single-start vs multi-start) e un commento su perché bounded in 1D è

competitivo.

3. Un confronto penalità vs proiezione sul box:

- quale rispetta il vincolo in modo “strutturale”?
- quale richiede più tuning (e perché)?

4. Per ciascun confronto con SciPy: riportare **la chiamata** (2–3 righe) e spiegare **che cosa si passa al solver**:

- funzione come `def f(x): ...` (o `lambda`),
- bracket / `x0`,
- bounds,
- output diagnostico (converged/success, iterations/message).

Sintassi (minima) per passare funzioni a SciPy

1) Caso 1D: una variabile reale x (root-finding / minimo 1D)

Qui x è un numero (float).

```
from scipy import optimize

def f(x):
    return x**3 - 2*x + 2

# zero con bracket (robusto)
sol = optimize.root_scalar(f, bracket=(-2, 0), method="brentq")
print(sol.root)

# minimo su intervallo
xmin = optimize.fminbound(f, -2, 0)
print(xmin)
```

2) Parametri extra in 1D: usare `args=...`

Esempio: $f(x; a, b) = ax^2 + b$.

```
from scipy import optimize

def f(x, a, b):
    return a*x**2 + b

# passa a e b con args
sol = optimize.root_scalar(f, bracket=(-2, 2), args=(1.0, -1.0), method="brentq")
print(sol.root)
```

Nota: `args` deve essere una tupla, anche con un solo parametro: `args=(a,)`.

3) Caso multivariato: **x** e un array (minimize)

Qui SciPy chiama la tua funzione passando **x** come array NumPy (anche se `d=1`).

```
import numpy as np
from scipy import optimize

def f(x):
    # x e un array: x[0], x[1], ...
    return (x[0]-1.0)**2 + (x[1]+2.0)**2

x0 = np.array([0.0, 0.0])

res = optimize.minimize(f, x0)
print(res.x, res.fun, res.success)
```

4) Parametri extra in multivariato: **args=...**

Esempio: $f(x; \mu) = (x_0 - 1)^2 + \mu(x_1 + 2)^2$.

```
import numpy as np
from scipy import optimize

def f(x, mu):
    return (x[0]-1.0)**2 + mu*(x[1]+2.0)**2

x0 = np.array([0.0, 0.0])

res = optimize.minimize(f, x0, args=(10.0,))
print(res.x, res.fun)
```

5) Vincoli box (bounds) in minimize

Esempio: minimizzare $(x - 3)^2$ con $x \in [0, 2]$.

```
from scipy import optimize

def f(x):
    return (x[0]-3.0)**2

x0 = [1.5]
bounds = [(0.0, 2.0)]

res = optimize.minimize(f, x0, method="L-BFGS-B", bounds=bounds)
print(res.x, res.fun, res.success)
```